

# Formato de Archivos: Binarios y de Texto

Apuntes de clase, organizados y ampliados

---

Sesiones: 8/10/2026 y 8/11/2026

Documento reorganizado a partir de notas originales de clase, con subdivisión temática y ampliación de conceptos clave.

## CONTENIDO

1. Estrategias de almacenamiento: tamaño fijo vs. tamaño variable
2. Repaso de archivos binarios
3. Fundamentos de los archivos de texto
4. Codificación de caracteres: de ASCII a Unicode
5. Unicode y UTF-8/16/32 en profundidad
6. Cómo se renderiza el texto: el rol de las fuentes (fonts)
7. Manejo de texto UTF-8 en C++
8. Binario vs. texto: comparación de ventajas y desventajas
9. Formatos de texto comunes: CSV, JSON y otros
10. ¿Cuándo usar cada tipo de archivo?
11. Temas pendientes de investigar

# 1. Estrategias de almacenamiento: tamaño fijo vs. tamaño variable

8/10/2026

## 1.1 Almacenamiento de tamaño fijo

Cuando cada registro ocupa siempre el mismo número de bytes, el manejo del archivo se simplifica considerablemente:

- **Deserialización más fácil:** como el tamaño de cada registro es conocido de antemano, leer un registro es simplemente leer un bloque de N bytes e interpretarlo, sin necesidad de "descubrir" dónde termina.
- **Cálculo de offset sencillo:** para acceder al registro `i`, basta con calcular `offset = i * tamaño_registro`. Esto permite acceso aleatorio en tiempo  $O(1)$  (equivalente a un arreglo en memoria, pero en disco).

### Problemas del almacenamiento de tamaño fijo

- **Desperdicio de espacio:** al reservar espacio para un registro, normalmente se dimensiona pensando en el caso más grande posible. Los registros que en la práctica son más pequeños dejan espacio sin usar.
- **Fragmentación interna:** es el nombre técnico de ese desperdicio — espacio reservado dentro de un bloque/registro que nunca se llega a utilizar. (Se distingue de la *fragmentación externa*, que ocurre entre bloques y no dentro de ellos.)

## 1.2 Almacenamiento de tamaño variable

Permitir que cada registro tenga un tamaño distinto resuelve el desperdicio de espacio, pero introduce nuevas complejidades:

- **Serialización y deserialización más complejas:** ya no basta con leer un bloque fijo; hay que codificar de alguna forma dónde termina cada registro (por ejemplo, guardando su longitud antes del contenido, o usando un delimitador).
- **Búsqueda más costosa:** como no se puede calcular un offset directamente, para llegar al registro `i` normalmente hay que leer secuencialmente todos los registros anteriores, ya que no se sabe de antemano dónde empieza cada uno.

### Soluciones al problema de la búsqueda

- **Índices:** una estructura auxiliar que guarda, para cada registro, la posición (offset) donde se encuentra en el archivo principal. Es común que los índices se implementen como archivos de tamaño *fijo* (por ejemplo, pares `[clave, offset]` de tamaño constante), combinando lo mejor de ambos mundos: registros variables en el archivo de datos, pero acceso rápido gracias al índice de tamaño fijo.
- **Llaves (keys):** identificadores únicos que permiten ubicar un registro dentro del índice.
- **Árboles B (B-trees):** estructura de datos balanceada, muy usada en bases de datos y sistemas de archivos, que organiza los índices en niveles para mantener búsquedas, inserciones y eliminaciones en tiempo  $O(\log n)$ , incluso con millones de registros en disco.

**Nota:** todo lo anterior se explicó tomando como referencia los **archivos binarios**, que se inspeccionan con un editor hexadecimal (por ejemplo `xxd`, `HxD` o `ImHex`). La pregunta que abre el resto del documento es: ¿qué cambia cuando hablamos de **archivos de texto**, que en cambio se leen con un editor de texto normal?

## 2. Repaso de archivos binarios

### 2.1 Almacenamiento y recuperación

- **Almacenamiento:** se escriben structs o tipos primitivos directamente a disco, típicamente con una llamada como `write()`. No hay conversión a texto; los bytes en memoria se copian casi tal cual al archivo.
- **Recuperación:** se lee con `read()` usando offsets fijos. Gracias a que los registros tienen tamaño fijo, el `seek` (posicionarse en un punto del archivo) es de complejidad  $O(1)$ .

### 2.2 Herramientas: el editor hexadecimal

Como los archivos binarios no representan texto legible, se inspeccionan con herramientas especializadas que muestran el contenido byte por byte en hexadecimal: `xxd` (línea de comandos), **HxD** (Windows) e **ImHex** (multiplataforma, con plantillas para interpretar estructuras).

### 2.3 Endianness

El *endianness* define el orden en que se almacenan los bytes de un valor multi-byte (como un entero de 32 bits):

- **Little-endian:** el byte menos significativo se guarda primero. El valor `0x00001234` se almacenaría en disco como los bytes `34 12 00 00`.
- **Big-endian:** el byte más significativo se guarda primero. El mismo valor se almacenaría como `00 00 12 34`.

**Por qué importa:** si un archivo binario se escribe en una arquitectura little-endian y se lee en una big-endian (o viceversa) sin tenerlo en cuenta, los números se interpretan incorrectamente. Es un tema crítico en portabilidad y también aparece en probabilidad/simulaciones cuando se procesan datos binarios generados en otra plataforma. La convención más extendida hoy en día (x86, ARM en modo normal) es little-endian.

### 2.4 Binario vs. Texto — primera comparación

| Binario                                                                            | Texto                                                                                           |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Bytes crudos, sin capa de codificación de caracteres — "se mira todo codificado"   | Se ve como texto legible por humanos (se puede inspeccionar con <code>cat</code> , por ejemplo) |
| Tipos nativos (int, float, struct) representados exactamente como están en memoria | Todo se representa como caracteres/símbolos codificados                                         |
| Acceso posicional / seek directo                                                   | Generalmente requiere parsing secuencial                                                        |
| No es legible sin una herramienta especializada                                    | Legible directamente con cualquier editor de texto                                              |
| Problemas de portabilidad: endianness, padding entre campos de un struct           | Portable siempre que se conozca la codificación (ej. UTF-8)                                     |

## 3. Fundamentos de los archivos de texto

---

8/10/2026 - 8/11/2026

### 3.1 La unidad mínima: el carácter

A diferencia del archivo binario, cuya unidad natural es el byte, la unidad mínima conceptual de un archivo de texto es el **carácter**. Esto incluye letras, números, símbolos de puntuación y también los **emojis**, que hoy en día se consideran símbolos Unicode igual que cualquier otro carácter.

**Importante:** un carácter *no necesariamente equivale a un byte*. Esa equivalencia solo era válida bajo la codificación ASCII original. En codificaciones modernas como UTF-8, un carácter puede ocupar entre 1 y 4 bytes.

### 3.2 Los archivos de texto siguen siendo datos binarios

Un punto clave que suele pasarse por alto: una vez almacenado en disco, un archivo de texto **sigue siendo, a nivel físico, una secuencia de bytes** — igual que un archivo binario. La diferencia no está en el disco, sino en el *acuerdo de interpretación*: existe una capa de codificación (un "diccionario" bytes ↔ caracteres) que el editor o programa usa para traducir esos bytes a símbolos legibles. Sin conocer esa codificación, un archivo de texto es, en la práctica, indistinguible de un binario cualquiera.

## 4. Codificación de caracteres: de ASCII a Unicode

---

La codificación de caracteres es, como se anotó en clase, "**la capa oculta**": el mecanismo invisible que traduce bytes a símbolos. Su evolución histórica responde directamente a la necesidad de representar cada vez más idiomas y símbolos.

### 4.1 ASCII (American Standard Code for Information Interchange)

- Fue el primer estándar ampliamente adoptado para codificar texto.
- Define **128 caracteres** (usa 7 bits), suficientes para el alfabeto inglés, dígitos, signos de puntuación y algunos caracteres de control.
- Bajo ASCII, la equivalencia *1 carácter = 1 byte* era exacta — de ahí que en su momento se asumiera esto como una regla general, algo que dejó de ser cierto con las codificaciones posteriores.

### 4.2 Latin-1 (ISO-8859-1)

- Extiende ASCII a **256 caracteres** (8 bits completos), agregando letras acentuadas y símbolos usados en varios idiomas de Europa occidental (por ejemplo español, francés, alemán).
- Sigue siendo de tamaño fijo (1 byte por carácter), pero insuficiente para cubrir todos los idiomas del mundo — sobre todo alfabetos no latinos (cirílico, árabe, chino, japonés, etc.).

### 4.3 El problema de la globalización

Cuando el software dejó de usarse solo en contextos angloparlantes y pasó a operar globalmente, la limitación de tener un único byte por carácter se volvió insostenible: no existe una codificación de 256 símbolos capaz de representar todos los alfabetos y símbolos del mundo a la vez. Esto llevó a la creación de un estándar universal.

### 4.4 Unicode: el estándar universal

**Unicode** es el estándar que asigna un **código único (code point)** a cada carácter o símbolo existente — desde letras de cualquier alfabeto hasta emojis y símbolos técnicos. Actualmente cubre más de un millón de puntos de código posibles.

**Distinción clave (subrayada explícitamente en clase):** Unicode y UTF **no son lo mismo**.

- **Unicode** es el estándar que define *qué* código le corresponde a cada carácter (por ejemplo, la letra "ñ" tiene el code point `U+00F1` ).
- **UTF (Unicode Transformation Format)** es el método concreto que se usa para *codificar* ese código en bytes reales dentro de un archivo. UTF-8, UTF-16 y UTF-32 son las tres variantes principales.

## 5. Unicode y UTF-8/16/32 en profundidad

---

### 5.1 UTF-8

- Es **compatible con ASCII tradicional**: los primeros 128 caracteres se codifican exactamente igual que en ASCII (1 byte), lo que facilita la retrocompatibilidad.
- Es un sistema de **codificación de ancho variable**: cada carácter puede ocupar entre 1 y 4 bytes (es decir, hasta 32 bits) según el símbolo que represente.
- Es, con diferencia, **el estándar dominante** en la web, sistemas operativos modernos y la mayoría de lenguajes de programación.

### 5.2 UTF-16

- Cubre también más de un millón de caracteres, pero usa unidades de 16 bits como base (con pares de unidades — "surrogate pairs" — para los caracteres fuera del plano básico).
- Es usado internamente por algunos sistemas y lenguajes (por ejemplo, Windows y Java lo usan como representación interna de strings), pero es menos común como formato de archivo/intercambio que UTF-8.

### 5.3 UTF-32

- Usa siempre 4 bytes fijos por carácter. Es el más simple de indexar (acceso directo por posición, como en tamaño fijo), pero el más costoso en espacio, ya que hasta los caracteres ASCII ocupan 4 bytes.

### 5.4 ¿Cómo distingue un programa entre UTF-8, UTF-16 y UTF-32?

Esta fue una pregunta abierta en los apuntes originales. En la práctica, un archivo de texto no "declara" su codificación dentro de los datos mismos salvo que se use alguna convención adicional, como:

- **BOM (Byte Order Mark)**: una secuencia de bytes especial al inicio del archivo que indica tanto la codificación UTF usada como, en el caso de UTF-16/32, el endianness. Por ejemplo, un archivo UTF-8 con BOM comienza con los bytes `EF BB BF`.
- **Metadatos externos**: declaración explícita en la cabecera del protocolo (por ejemplo, `Content-Type: text/html; charset=utf-8` en HTTP) o en el propio documento ( `<meta charset="utf-8">` en HTML).
- **Convención por defecto**: a falta de indicación, la inmensa mayoría de editores, terminales y sistemas asumen UTF-8 hoy en día, precisamente por ser el estándar dominante.

### 5.5 Ejemplo práctico: bytes crudos de "ñ" y "Ñ" en UTF-8

**De los apuntes originales:** se anotaron los bytes hexadecimales `C3 B1` y `C3 91` como ejemplo. Estos corresponden exactamente a la codificación UTF-8 de dos caracteres del español:

- `C3 B1` → carácter "ñ" (U+00F1), codificado en 2 bytes.
- `C3 91` → carácter "Ñ" (U+00D1), codificado en 2 bytes.

Esto ilustra de forma muy directa por qué "un carácter no es necesariamente un byte": en ASCII/Latin-1 estos caracteres ocupan 1 byte, pero en UTF-8 ocupan 2, porque están fuera del rango de los primeros 128 code points compatibles con ASCII.

## 6. Cómo se renderiza el texto: el rol de las fuentes (fonts)

---

### 6.1 Del código al dibujo en pantalla

**Renderizar** significa desplegar o dibujar en pantalla. El proceso completo, tal como se explicó en clase, tiene dos etapas separadas:

1. **Identificación del símbolo:** cada carácter tiene un código único (su code point Unicode). El decodificador lee los bytes del archivo, aplica la codificación correspondiente (UTF-8, etc.) y determina exactamente qué símbolo representan.
2. **Dibujado del carácter:** una vez identificado el código, el editor consulta una **fuentes (font)** para saber *cómo se ve* ese símbolo gráficamente. El editor, en el fondo, simplemente le pregunta al font: "para este código identificado, ¿qué forma tengo que dibujar?".

### 6.2 Tipos de fuentes

- **Fuentes vectoriales (Vector fonts) — ej. TrueType Font (TTF):** almacenan la representación gráfica de cada carácter como un conjunto de **operaciones de dibujo** (curvas, líneas), de forma similar a como funciona un archivo **SVG**. Al ser vectoriales, se pueden escalar a cualquier tamaño sin perder calidad.
- **Fuentes de mapa de bits (Bitmap fonts):** almacenan directamente una imagen (una cuadrícula de píxeles) para cada carácter, en lugar de instrucciones de dibujo. Su limitación principal es que **se distorsionan al cambiar de tamaño (resize)**, ya que no hay información vectorial que reescalar limpiamente — solo se estira o interpola la imagen original.

El mismo mecanismo aplica a los **emojis**: no son un caso especial a nivel de codificación, son simplemente otro code point Unicode más, y son los fonts (a menudo fuentes de emoji especializadas y a color) los que definen su representación gráfica final.

## 7. Manejo de texto UTF-8 en C++

### 7.1 `std::string` vs. codificación real

- `std::string` almacena texto, pero **no verifica ni impone ninguna codificación**: simplemente guarda la secuencia de bytes que se le asigna, sin saber (ni preguntarse) si esos bytes forman UTF-8 válido, Latin-1, o cualquier otra cosa.
- Existen librerías/recursos externos dedicados específicamente a **validar** que una secuencia de bytes sea UTF-8 válido, ya que el lenguaje por sí solo no lo garantiza.
- Quien realmente "maneja" la codificación al mostrar texto es el **editor o la terminal**: si logramos ver correctamente lo que escribimos (incluyendo tildes o emojis), es porque el editor/terminal está interpretando esos bytes como UTF-8 y renderizándolos con la fuente adecuada — no porque `std::string` haga ese trabajo.

### 7.2 `std::u8string` y `std::string_view`

Desde C++20, el lenguaje ofrece `std::u8string`, un tipo pensado específicamente para **garantizar** (a nivel de tipo) que los datos almacenados están codificados en UTF-8. `std::string_view`, por su parte, no crea una copia nueva del texto: es solo una **referencia de solo lectura** a datos ya existentes en memoria, lo cual evita copias innecesarias.

```
#include <iostream>
#include <string>
#include <string_view>    // no crea un registro nuevo, sino una
                        // referencia a un registro existente

int main() {
    std::u8string str = u8"Hello World 🤔 🤔";

    std::string_view sv(
        reinterpret_cast<const char*>(str.data()),
        str.size()
    );

    std::cout << sv << '\n';
    std::cout << "Length: " << str.length() << '\n';
}
```

**Qué garantiza este código y qué no:** `std::u8string` nos asegura que los datos están codificados como UTF-8 (es, en esencia, un compromiso de tipo — "prometo que esto es UTF-8"). No hace magia adicional: para poder imprimirlo con `std::cout`, hay que reinterpretar los bytes como `char` normales a través de un `string_view`, porque los operadores de flujo estándar no tienen una sobrecarga directa para `char8_t`.



## 8. Binario vs. texto: comparación de ventajas y desventajas

### 8.1 Ventajas y desventajas de los archivos binarios

| Ventajas                                                                                                  | Desventajas                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compacto:</b> sin overhead de codificación de caracteres, ocupa menos espacio                          | <b>No legible</b> sin herramientas especializadas                                                                                               |
| <b>I/O rápido:</b> la serialización y deserialización son directas (copia de bytes), sin parsing de texto | <b>Portabilidad limitada:</b> problemas de endianness y de padding entre plataformas/compiladores                                               |
| <b>Tipos exactos:</b> los datos se representan tal como están en memoria (int, float, struct)             | <b>Versionado difícil:</b> cuando el formato del struct cambia entre versiones del programa, hay que gestionar explícitamente la compatibilidad |
| <b>Seek directo:</b> acceso aleatorio $O(1)$ con registros de tamaño fijo                                 | <b>Sin schema:</b> es complicado inferir o comunicar la estructura de los datos solo a partir del archivo                                       |

### 8.2 Ventajas y desventajas de los archivos de texto

| Ventajas                                                                                                                                         | Desventajas                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Legible para humanos</b> directamente, sin herramientas especiales                                                                            | <b>Mayor tamaño</b> en disco que su equivalente binario                                                                                        |
| <b>Portabilidad total</b> entre sistemas operativos y arquitecturas (siempre que se respete la codificación)                                     | <b>Parsing más lento:</b> hay que interpretar caracteres y convertirlos a tipos (deserialización más costosa)                                  |
| <b>Interoperabilidad</b> entre lenguajes y sistemas distintos                                                                                    | <b>Ambigüedad de tipos:</b> por ejemplo, el texto <code>"123"</code> puede ser un número o una cadena — el formato no lo distingue por sí solo |
| <b>Control de versiones</b> amigable: se pueden usar manejadores de código como Git para visualizar los cambios línea por línea (diffs legibles) | <b>Sin soporte nativo</b> para tipos de datos complejos; todo debe convertirse a representación textual                                        |

**Detalle adicional sobre portabilidad de texto:** incluso entre archivos de texto existen pequeñas diferencias de portabilidad, como el carácter de salto de línea: Unix/Linux/macOS usan un único carácter `\n` (LF), mientras que Windows usa dos caracteres `\r\n` (CRLF) — por eso se dice que "el código Unix de salto de línea es más barato" (ocupa menos). Esta diferencia es una fuente común de problemas al compartir archivos de texto entre sistemas operativos distintos.

## 9. Formatos de texto comunes: CSV, JSON y otros

---

### 9.1 CSV (Comma-Separated Values)

Formato tabular de texto plano donde cada línea representa un registro y los campos dentro de esa línea están separados por un delimitador (típicamente una coma).

- **Uso principal:** importación y exportación de datos tabulares entre sistemas (hojas de cálculo, bases de datos, etc.).
- **Problema principal — falta de tipos:** cada campo es simplemente texto; el formato no especifica si un valor es un entero, un decimal, una fecha o una cadena. Esa interpretación queda por completo del lado de quien lee el archivo.
- **Solución común:** acompañar el archivo CSV de datos con un archivo adicional (a veces llamado "schema" o diccionario de datos) que documenta explícitamente qué tipo y formato tiene cada columna, evitando así la ambigüedad.
- **Delimitadores no estandarizados:** aunque la coma es lo más común, también se usan punto y coma, tabulaciones (formato TSV) u otros caracteres, lo que puede generar incompatibilidades entre herramientas si no se especifica claramente cuál se está usando.
- **Librerías de parsing:** dado que el CSV parece simple pero tiene muchos casos borde (comillas, delimitadores dentro de un campo, saltos de línea dentro de un campo entrecomillado), en la práctica casi siempre conviene usar una librería especializada para leerlo/escribirlo en vez de hacer un split manual por comas.

### 9.2 JSON (JavaScript Object Notation)

Mencionado en los apuntes originales como uno de los "formatos que sirven". A diferencia del CSV, JSON sí permite representar estructuras jerárquicas (objetos anidados, listas) y distingue nativamente entre algunos tipos básicos como cadenas, números, booleanos, listas y objetos — lo cual mitiga (aunque no elimina del todo) el problema de ambigüedad de tipos que tiene el CSV.

### 9.3 Otros formatos de configuración

En la sección sobre casos de uso, los apuntes mencionan además **YAML** y **TOML** como formatos de texto pensados específicamente para archivos de configuración legibles y editables por personas (a diferencia de JSON, que es más común para intercambio de datos entre programas), junto con el clásico formato **INI**.

## 10. ¿Cuándo usar cada tipo de archivo?

---

### 10.1 Cuándo conviene usar formato binario

- **Rendimiento crítico:** juegos, simulaciones físicas, procesamiento en tiempo real, donde cada milisegundo de I/O importa.
- **Datos multimedia:** imágenes en formatos raw (PNG/JPEG), audio (por ejemplo PCM sin comprimir), video — datos que no tienen una representación textual natural.
- **Protocolos de red:** cabeceras TCP/IP y protocolos binarios diseñados para eficiencia, como MessagePack o Protobuf.
- **Gran volumen de datos:** cuando se manejan millones de registros y el tamaño en disco/memoria es una restricción real.

### 10.2 Cuándo conviene usar formato de texto

- **Archivos de configuración:** JSON, YAML, TOML, INI — formatos pensados para que el usuario final pueda editarlos directamente.
- **Intercambio de datos:** APIs REST, exportaciones de datos, integración entre sistemas distintos que necesitan un formato común y legible.
- **Logs de sistema:** cuando un operador humano necesita poder leer el log directamente, sin pasos intermedios de decodificación.
- **Colaboración y control de versiones:** cuando importa poder ver el *diff* (los cambios línea por línea) en Git, o facilitar la revisión de código entre pares — algo prácticamente imposible de hacer de forma legible con archivos binarios.

**Regla general para retener:** la decisión entre binario y texto casi siempre se reduce a una tensión entre **eficiencia** (velocidad, tamaño, acceso directo) y **legibilidad/interoperabilidad** (humanos, control de versiones, portabilidad entre sistemas). Muchos sistemas reales usan ambos: por ejemplo, datos binarios para el almacenamiento interno de alto rendimiento, y exportaciones en texto (CSV/JSON) para intercambio con el exterior.

## 11. Temas pendientes de investigar

---

Los apuntes originales señalan explícitamente algunos temas que quedaron abiertos para profundizar después de clase:

**Struct padding.** Se refiere al espacio de relleno que el compilador inserta entre los campos de un struct para respetar los requisitos de alineación de memoria de cada tipo de dato (por ejemplo, un `int` de 4 bytes suele necesitar empezar en una dirección múltiplo de 4). Este padding afecta directamente el tamaño real de un struct al serializarlo a binario, y es una de las razones detrás del problema de portabilidad mencionado en la sección 8.1 — el mismo struct puede tener padding distinto según el compilador, la plataforma o las opciones de compilación usadas.

**Soporte de UTF para C++.** Revisar en más detalle qué ofrece la librería estándar de C++ (tipos como `char8_t` / `std::u8string`, `char16_t` / `std::u16string`, `char32_t` / `std::u32string`, y las facilidades de `<codecvt>` / librerías externas) para trabajar de forma segura con las distintas codificaciones UTF, más allá de lo cubierto en la sección 7.